# From Pixels to Land Use: How We Broke the 99% Barrier on EuroSAT

By: Arnau Arevalo, Rowan Becker, Siyoung Kim, Alfred Morales

## Goals

Our goal with this project was to use machine learning to first match, then improve upon, existing land use classifiers. The main datasets used were EuroSAT [1, 2] and UC Merced Land Use [3], along with supplemental imagery from Microsoft Planetary Computer [4].

## What is Land Use Classification?

> “Land use” is the term used to describe the human use of land. It represents the economic and cultural activities (e.g., agricultural, residential, industrial, mining, and recreational uses) that are practiced at a given place. [5]

In land use classification, the idea is to train a model to look at a satellite or aerial image of land, and label it as one of a list of categories. EuroSAT, for example, classifies images into one of ten categories:

- Annual Crop
- Forest
- Herbaceous Vegetation
- Highway
- Industrial
- Pasture
- Permanent Crop

- Residential
- River
- Sea Lake

# The Datasets We Worked With

| | EuroSAT RGB | EuroSAT Multi-Spectrum | UC Merced |
|---|---|---|---|
| Source | Sentinel-2 | Sentinel-2 | USGS Aerial |
| Classes | 10 | 10 | 21 |
| Images | 27,000 | 27,000 | 2,100 |
| Resolution | 64×64 | 64×64 | 256×256 |
| Pixel Size | 10 meters | 10 meters | 1 foot |

# First Attempts Using ResNet18

The authors of EuroSAT created a model that was able to achieve 98.57% accuracy in classifying satellite images. First, we just wanted to try and replicate that using the EuroSAT RGB dataset as input (RGB allows for a simpler pipeline as ResNet18 takes 3 channels as input). We used a pretrained ResNet18 [6], a convolutional neural network (CNN) trained on ImageNet [7] with ~11.7 million parameters. To adapt it for this specific dataset, we fine-tuned it like so:

1. Replace the last fully connected layer with an empty fully connected layer with an output size of 10 to match EuroSAT's 10 classes.

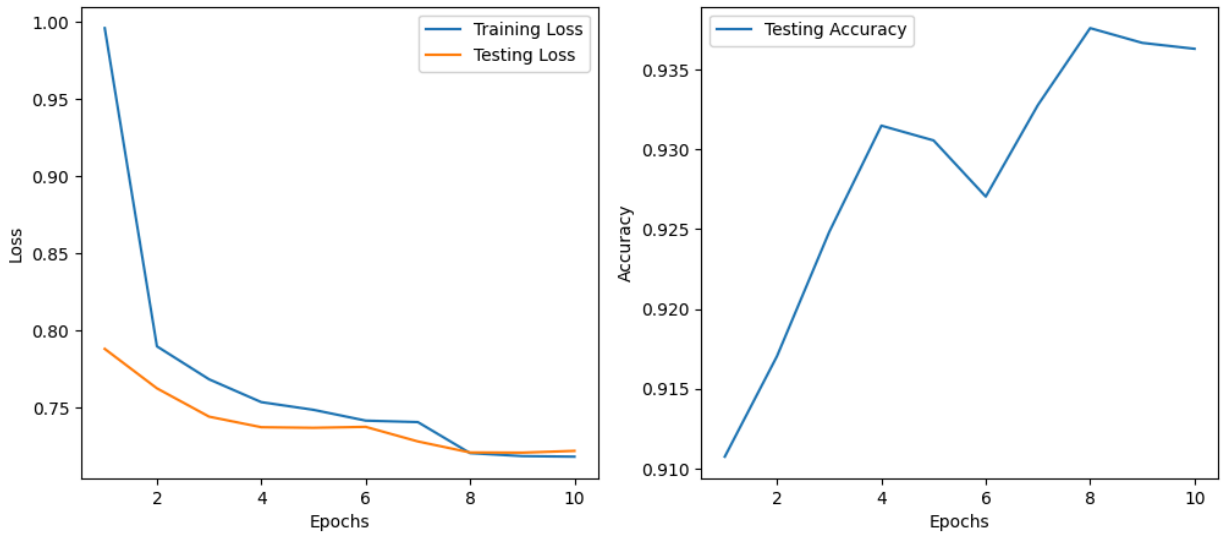
```
fc_input_features = model.fc.in_features
model.fc = nn.Linear(fc_input_features, 10)
```

2. Freeze all layers except for the last one, and train the model for 10 epochs.

```
# Freeze all layers
for param in model.parameters():
    param.requires_grad = False
```

```python
# Unfreeze the last layer
for param in model.fc.parameters():
    param.requires_grad = True

# Train
torch.manual_seed(123)
optimizer = optim.Adam(model.fc.parameters(), lr=0.001)
epochs = 10
train(model, train_loader, test_loader, optimizer, epochs)
```

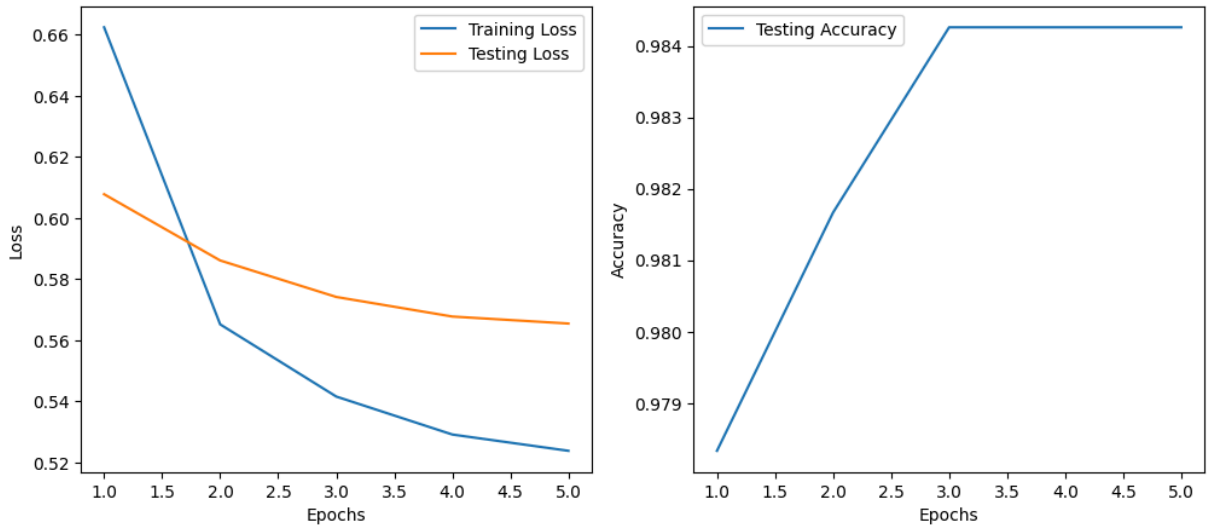



3. Unfreeze all layers and train the whole network for 5 more epochs.

```python
# Unfreeze all layers
for param in model.parameters():
    param.requires_grad = True


# Redefine optimizer to include all parameters
optimizer = optim.Adam(model.parameters(), lr=0.0001)
```

```
# Train
optimizer = optim.Adam(model.parameters(), lr=0.00005)
epochs = 5
train(model, train_loader, test_loader, optimizer, epochs)
```




```
Training Accuracy: 100.00%
Testing Accuracy: 98.43%
```

Not bad, with just a fine-tuned ResNet18 we can almost match the authors' accuracy. Now our goals become two-fold. First, can we improve accuracy? Second, can we improve sample efficiency (getting the same accuracy despite training on fewer of the labelled satellite images in EuroSAT)?

# Label Smoothing

When training a standard classifier, we use what's called "hard" targets - a one-hot encoded vector where the correct class gets a value of 1 and all other classes get 0. For example, if an image is a forest, the target looks like this: [0, 1, 0, 0, 0, 0, 0, 0, 0, 0]. This tells the model: "Be 100% certain this is a forest and absolutely nothing else."

The problem is that this encourages the model to become overconfident. It learns to push its predicted probability for the correct class toward 1.0, even when the training example might be ambiguous or contain elements of multiple classes. A forest tile might have a small river running through it. An industrial tile might have

a patch of grass. But hard labels ignore these nuances, forcing the model to pretend the image is unmixed.

This overconfidence leads directly to overfitting. The model starts memorizing training examples rather than learning generalizable patterns. You can see this in our baseline results: training accuracy reached 99.87% while test accuracy plateaued at 98.30%, a gap of 1.57 percentage points. The model was too confident in its training predictions and struggled to generalize to new data.

## The Solution

Label smoothing addresses this by softening the target distribution. Instead of asking for 100% confidence, we ask for something more realistic. The formula is simple:

```
smoothed_target = one_hot * (1 - smoothing) + (smoothing / num_classes)
```

With our settings - smoothing factor of 0.1 and 10 classes - the math works out as:

- Correct class: 1 × (1 - 0.1) + 0.1/10 = 0.9 + 0.01 = 0.91
- Every other class: 0 × 0.9 + 0.01 = 0.01

So the target becomes: [0.01, 0.91, 0.01, 0.01, 0.01, 0.01, 0.01, 0.01, 0.01, 0.01] The model is now only asked to be 91% confident about the correct class. The remaining 9% probability gets spread evenly across all other classes. This small change has profound effects on training dynamics.

## Why This Works

1. Prevents Extreme Prediction

Without label smoothing, the model can push its logits to arbitrarily large values to drive the correct class probability toward 1.0. With smoothing, there's a finite target - 0.91 - so the model stops once it's "close enough." This prevents the weights from growing too large and keeps the model in a region where it generalizes better.

2. Acts as a Regularizer

Label smoothing is mathematically equivalent to adding a divergence penalty between the model's predicted distribution and a uniform distribution. This penalty prevents the model from becoming too confident and effectively adds noise to the training process, similar to dropout or weight decay. It forces the model to find more robust features that work well even when we don't let it fully commit.

## The Results

| Metric | Without Label Smoothing | With Label Smoothing | Change |
|---|---|---|---|
| Training Accuracy | 99.87% | 100.00% | +0.13% |
| Testing Accuracy | 98.30% | 98.74% | +0.44% |
| Overfitting Gap | 1.57% | 1.26% | -0.31% |

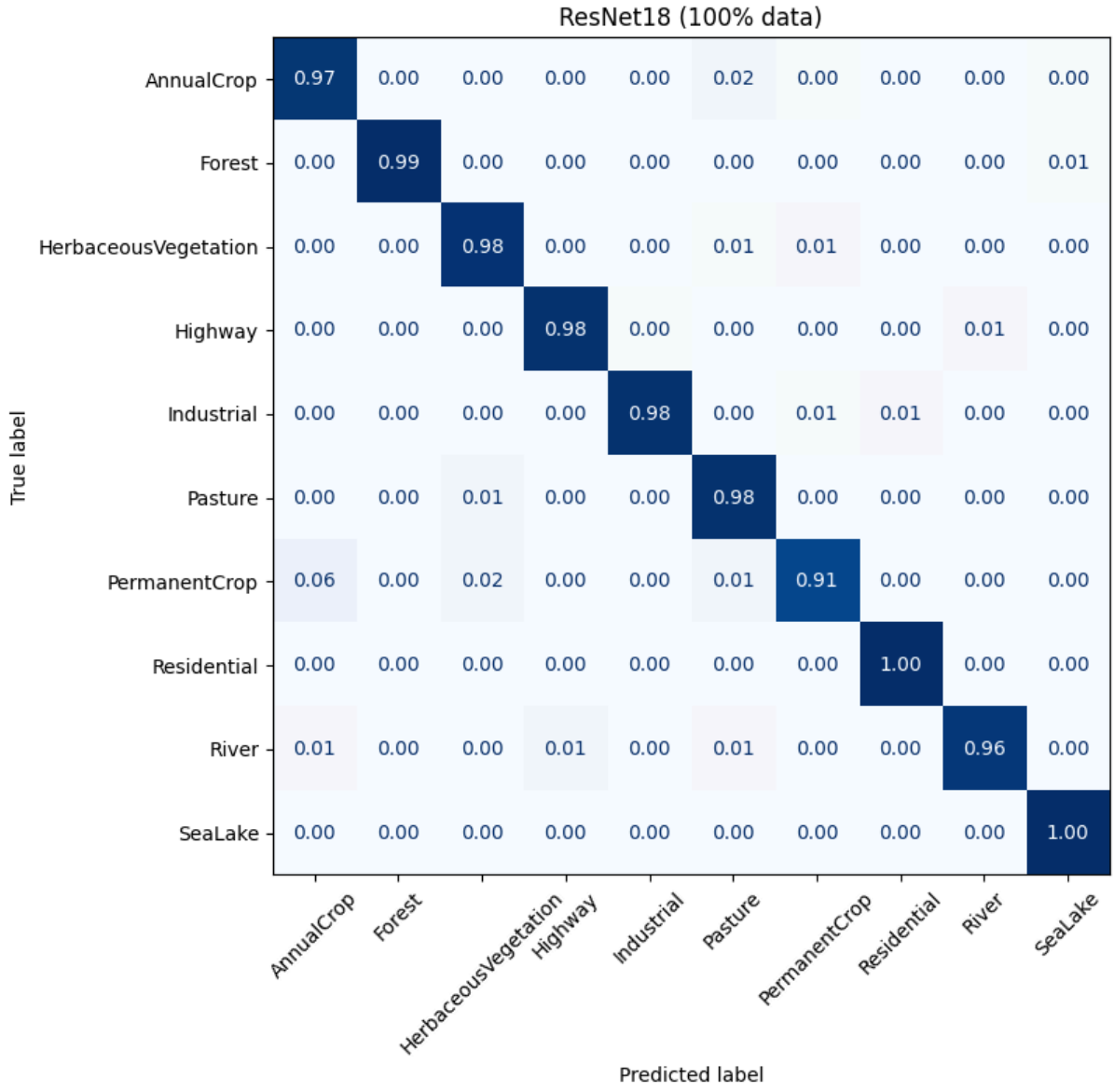
Training accuracy actually increased to 100%. The explanation is that label smoothing stabilized training, allowing the optimizer to find a better minimum without the destabilizing effects of pushing logits to extreme values.

More importantly, test accuracy improved by 0.44 percentage points. On a test set of roughly 5,400 images (20% of 27,000), this means our model correctly classified about 24 more images that it had never seen before. The overfitting gap shrank by nearly 20% (from 1.57% to 1.26%), confirming that label smoothing successfully reduced overfitting.

# Improving Further

Around this point, we were struggling to push past ~98.7% with the RGB and ~99.0% with the Multi-Spectrum datasets. So we looked into incorporating better data as input. Because the EuroSAT images are geo-referenced, we can find the coordinates of each image. Unfortunately, we could not find free historical (EuroSAT was taken somewhere in 2016 or 2017), high resolution, international data. So increasing the resolution of the images themselves was out of the question.

In particular, the models misclassified permanent crops as some of the other greenery classes.



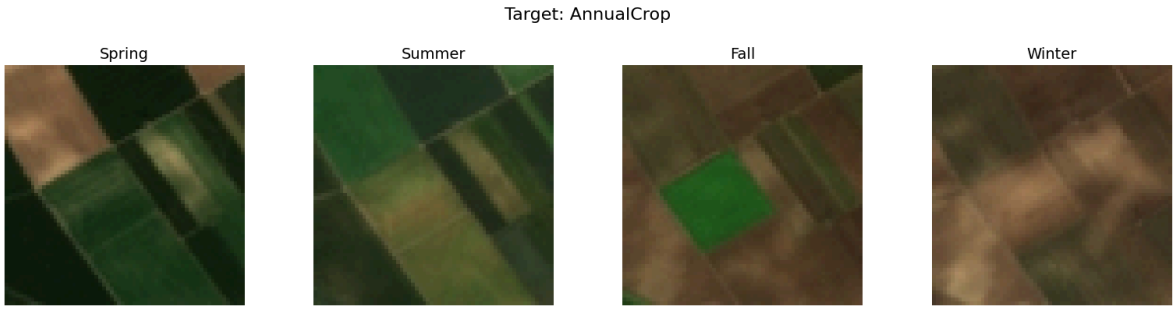

How can we increase the useful information and context that the models have? We decided to add two new sets of data.

1. **Spatial context (adjacent tiles)**: We wrote a script to download the 8 surrounding neighbor tiles for all 27,000 images. The idea was to give the model a more macro view of the areas (e.g., “If this green patch is surrounded by highways, it probably isn’t a deep forest”).


💡 *Contains modified Copernicus Sentinel data [2026]*

2. **Temporal context**: To directly address the permanent crop issue, we added one image from each season in 2017 for every image in the EuroSAT dataset.

> Our thinking was that permanent crops tend to have green or non-plowed fields year round, while annual crops have more seasonality.




> 💡 *Contains modified Copernicus Sentinel data [2026]*

Because we are only using these two sets of images as extra context, we don't need to label them, so it don't add any extra labeling time to dataset curation.

## Dual Branch ResNet18

We couldn't throw all this data into a standard ResNet18. Because the adjacent tiles represent distinct geographic areas, we needed the model to understand spatial boundaries. To address this, we split it into two branches:

1. Deep Temporal and Spectral Context: A ResNet modified to accept a 25-channel tensor (13 multi-spectral bands from the EuroSAT MS dataset + 12 temporal RGB channels, 3 RGB channels x 4 seasons). This learns how a specific 640m square reflects infrared light and changed colors over the year.
2. Wide Context: A standard ResNet that took a 192×192 stitched grid of the original tile and its 8 neighbors as input to learn spatial relationships.

We combined the 512 dimension outputs from both branches into a final fully connected layer to make the prediction.

First, we built a custom PyTorch Dataset to fetch and format the two data streams for each image.

```
class EuroSAT_Fusion_Dataset(Dataset):
    def __init__(self, cache_file, ms_dir, rgb_dir, seasons_d
```

```
ir, adjacents_dir, transform_wide=None):
        self.ms_dir = ms_dir
        self.rgb_dir = rgb_dir
        self.seasons_dir = seasons_dir
        self.adjacents_dir = adjacents_dir
        self.transform_wide = transform_wide

        with open(cache_file, 'r') as f:
            full_cache = json.load(f)
        self.tiles = [k for k in full_cache.keys() if "_gener
ated_" not in k]
        self.cache = full_cache

        self.classes = sorted(["AnnualCrop", "Forest", "Herba
ceousVegetation", "Highway",
                               "Industrial", "Pasture", "Perm
anentCrop", "Residential",
                               "River", "SeaLake"])
        self.class_to_idx = {cls_name: i for i, cls_name in e
numerate(self.classes)}

        self.grid_map = {
            'NW': (0, 0), 'N': (64, 0), 'NE': (128, 0),
            'W': (0, 64), 'C': (64, 64), 'E': (128, 64),
            'SW': (0, 128), 'S': (64, 128), 'SE': (128, 128)
        }

    def __len__(self):
        return len(self.tiles)

    def __getitem__(self, idx):
        target_tile = self.tiles[idx]
        class_name = target_tile.split('_')[0]
        tile_clean = target_tile.replace('.jpg', '').replace
('.tif', '')
        label = self.class_to_idx[class_name]
```

```
        # Branch 1: Deep tensor
        deep_channels = []
        ms_path = os.path.join(self.ms_dir, class_name, f"{ti
le_clean}.tif")
        with rasterio.open(ms_path) as src:
            ms_data = src.read().astype(np.float32) / 10000.0
            deep_channels.append(torch.tensor(ms_data))

        for season in ["Spring", "Summer", "Fall", "Winter"]:
            season_path = os.path.join(self.seasons_dir, clas
s_name, tile_clean, f"{season}.jpg")
            if os.path.exists(season_path):
                img_pil = Image.open(season_path).convert("RG
B").resize((64, 64))
                img = np.array(img_pil).astype(np.float32) /
255.0
            else:
                img = np.zeros((64, 64, 3), dtype=np.float32)

            img = np.transpose(img, (2, 0, 1))
            deep_channels.append(torch.tensor(img))

        x_deep = torch.cat(deep_channels, dim=0)

        # Branch 2: Wide tensor
        canvas = Image.new('RGB', (192, 192), (0, 0, 0))

        center_path = os.path.join(self.rgb_dir, class_name,
f"{tile_clean}.jpg")
        if os.path.exists(center_path):
            center_img = Image.open(center_path).convert("RG
B").resize((64, 64))
            canvas.paste(center_img, self.grid_map['C'])

        neighbors = self.cache[target_tile].get("neighbors",
```

```
{})
        for direction, filename in neighbors.items():
            adj_path = os.path.join(self.adjacents_dir, filen
ame)
            if os.path.exists(adj_path):
                adj_img = Image.open(adj_path).convert("RG
B").resize((64, 64))
                canvas.paste(adj_img, self.grid_map[directio
n])

        if self.transform_wide:
            x_wide = self.transform_wide(canvas)
        else:
            x_wide = transforms.ToTensor()(canvas)

        return x_deep, x_wide, label
```

Then, we modified the first convolutional layer of Branch 1 to take 25 input channels, and then later concatenate the two 512 dimensional embeddings.

```
class DualBranchResNet(nn.Module):
    def __init__(self, num_classes=10, dropout_rate=0.5):
        super(DualBranchResNet, self).__init__()

        pretrained_b1 = models.resnet18(weights=models.ResNet
18_Weights.IMAGENET1K_V1)
        self.branch1 = pretrained_b1
        self.branch1.conv1 = nn.Conv2d(25, 64, kernel_size=7,
stride=2, padding=3, bias=False)
        self.branch1.fc = nn.Identity()

        pretrained_b2 = models.resnet18(weights=models.ResNet
18_Weights.IMAGENET1K_V1)
        self.branch2 = pretrained_b2
        self.branch2.fc = nn.Identity()
```

```
        self.fusion = nn.Sequential(
            nn.Dropout(dropout_rate),
            nn.Linear(1024, num_classes)
        )

    def forward(self, x_deep, x_wide):
        emb1 = self.branch1(x_deep)
        emb2 = self.branch2(x_wide)
        fused = torch.cat((emb1, emb2), dim=1)
        out = self.fusion(fused)
        return out
```

## Overfitting and Tuning

As is common, we quickly ran into an overfitting problem. With so much data, the model was simply overfitting and stopped generalizing any further. However, because a top-down image of a farm is still a farm no matter which direction you rotate it, by randomly rotating images, smoothing labels, and adding L2 regularization, we were able to find further small improvements.

After some hyperparameter tuning, we were able to achieve 99.25% accuracy with this improved dual branch ResNet model.

## The Limits of ResNet18 and Trying ConvNeXT [8]

Most of our ResNet18 attempts at this point were able to reach 99.0% to 99.2% accuracy, but not any further. I suspect this is more due to dataset noise and label imperfections holding things back, but nonetheless, if this is the limit of ResNet18 then the next step is to try another model. ResNet18 is almost a decade old at this point, and there have been many more modern CNNs made that incorporate newer data and better techniques. One such model is ConvNeXT-Tiny, a ~28.6 million parameter CNN created in 2022.

We adopted the same dual branch architecture that we used for ResNet, but we did have to make three distinct changes due to ConvNeXt's different architecture.

1. ResNet uses a 7×7 convolution to initially look at the image, but ConvNeXt uses a 4×4 convolution with a stride of 4, so we have to be careful to keep

those Conv2d parameters the same when we replace it with one that takes 25 input channels.

2. ConvNeXt classifies with a three step sequence (LayerNorm2d → Flatten → Linear). We only want to replace the Linear layer with our 10 class output Linear layer.
3. The fusion head we make from the two branch output vectors needs to have an input size of 1536 because ConvNeXt-Tiny outputs a 768 dimensional vector as opposed to ResNet18's 512 dimensional vector.

```
class DualBranchConvNeXt(nn.Module):
    def __init__(self, num_classes=10, dropout_rate=0.5):
        super(DualBranchConvNeXt, self).__init__()

        # Branch 1
        self.branch1 = models.convnext_tiny(weights=models.Co
nvNeXt_Tiny_Weights.DEFAULT)
        self.branch1.features[0][0] = nn.Conv2d(25, 96, kerne
l_size=4, stride=4)

        self.branch1.classifier[2] = nn.Identity()

        # Branch 2
        self.branch2 = models.convnext_tiny(weights=models.Co
nvNeXt_Tiny_Weights.DEFAULT)
        self.branch2.classifier[2] = nn.Identity()

        # Fusion Head
        self.fusion = nn.Sequential(
            nn.Dropout(dropout_rate),
            nn.Linear(1536, num_classes)
        )

    def forward(self, x_deep, x_wide):
        emb1 = self.branch1(x_deep)
        emb2 = self.branch2(x_wide)
```

```
        fused = torch.cat((emb1, emb2), dim=1)
        out = self.fusion(fused)
        return out
```

We were able to reach 99.33% accuracy, the highest accuracy we were able to achieve during this project.

While our Dual Branch ConvNeXt pushed us close to the limits of CNNs, we were also curious how a fundamentally different architecture (Vision Transformers) would handle the baseline data.

# A Different Architecture

## Vision Transformers

So far we've been using ResNet18, a CNN. But CNNs aren't the only option in computer vision. Vision Transformers (ViT) [9] take a fundamentally different approach. Instead of sliding small filters across the image, a ViT chops the image into 16×16 patches and lets every patch attend to every other patch from the very first layer. That global view *might* be useful for satellite tiles, where land use depends on context across the whole tile, not just local texture.

Note that ViTs are quite data-hungry. The original ViT paper showed they underperform CNNs unless trained on very large datasets. EuroSAT only has 27,000 tiles. Would a pretrained ViT-Small still hold up?

We used `vit_small_patch16_224` from the `timm` library [10], pretrained on ImageNet, comparable in spirit to our ResNet18 setup:

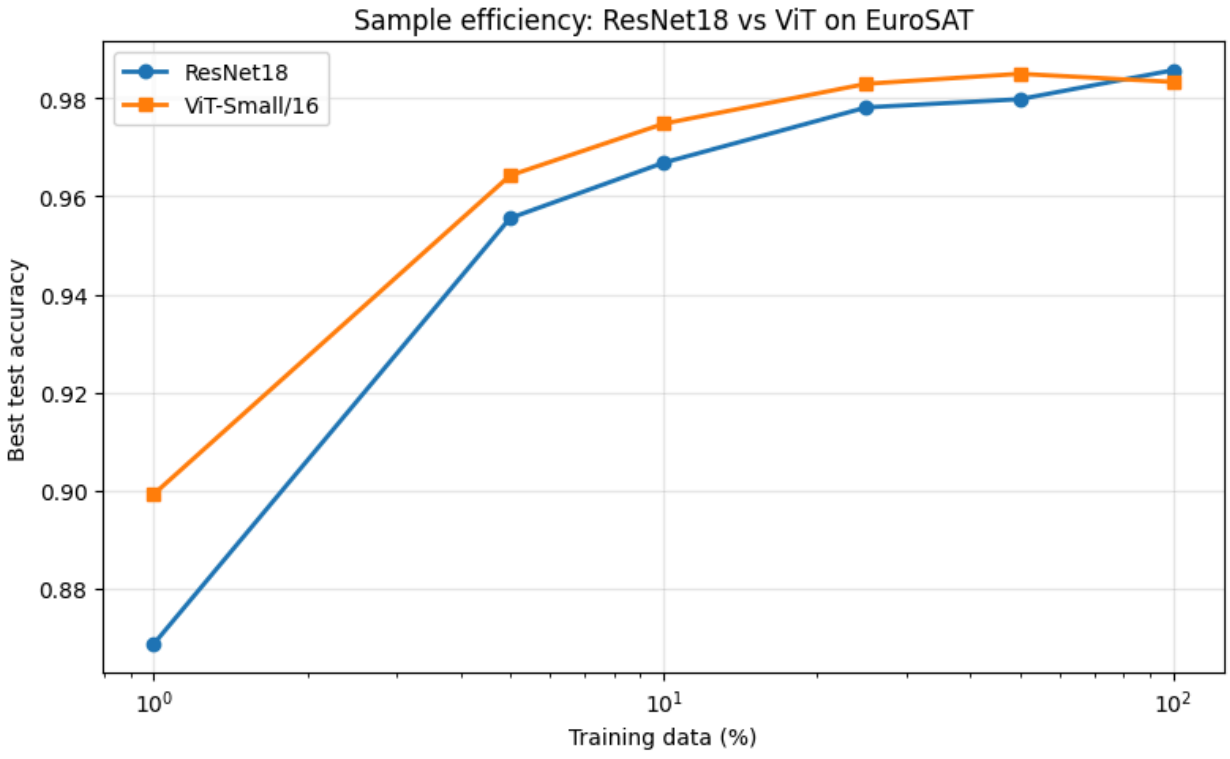
```
import timm
vit = timm.create_model('vit_small_patch16_224', pretrained=T
rue, num_classes=10)
```

# Sample Efficiency

## How Does Each Model Scale With Data?

This is where it gets interesting. Rather than just comparing both models at full data, we trained each on increasing fractions of the training set: 1%, 5%, 10%, 25%, 50%, and 100%. The question isn't just "which model is better", rather it's "how does each model's performance scale with data?”

| Fraction | ResNet18 | Vit-Small | Δ(Vit-ResNet) |
|---|---|---|---|
| 1% | 0.8687 | 0.8993 | +3.06% |
| 5% | 0.9556 | 0.9643 | +0.87% |
| 10% | 0.9669 | 0.9748 | +0.79% |
| 25% | 0.9781 | 0.9830 | +0.48% |
| 50% | 0.9798 | 0.9850 | +0.52% |
| 100% | 0.9857 | 0.9833 | -0.24% |




Two things stand out.

1. ViT-Small consistently *beats* ResNet18 at low data by over 3 percentage points at 1% data (just 216 training samples!).
2. However, that advantage shrinks monotonically as data grows, and at 100% data ResNet18 actually edges out ViT by a hair.

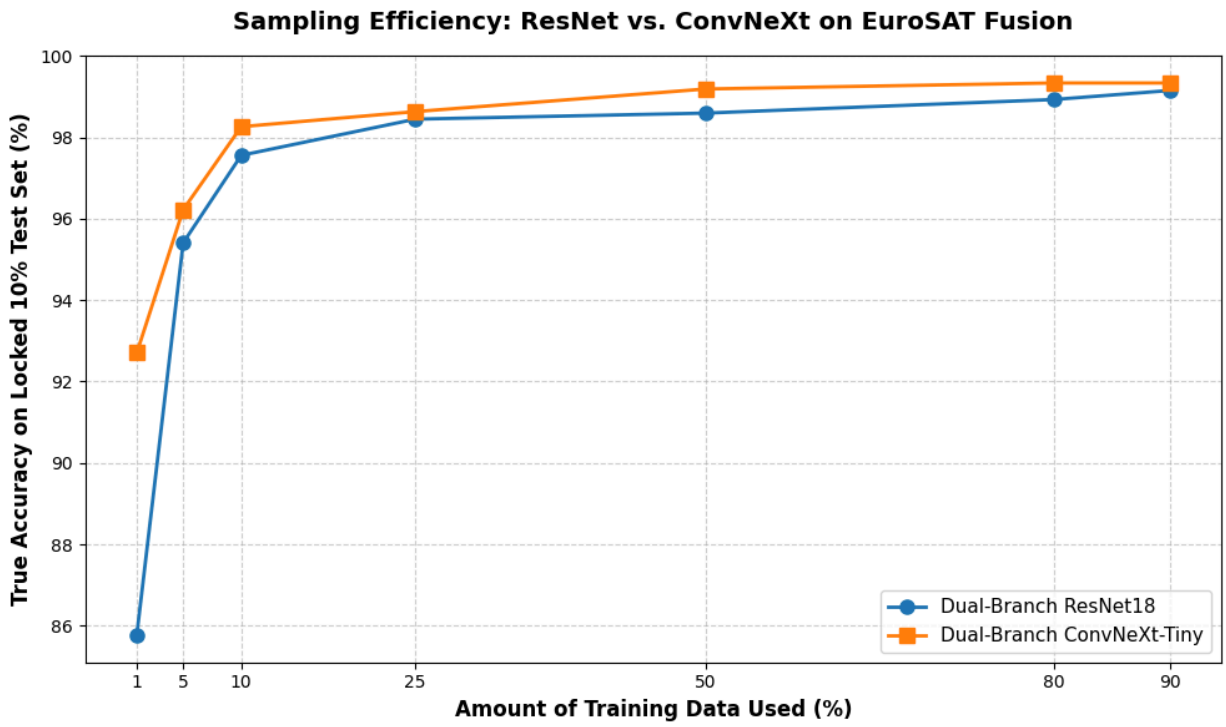
This is a bit of a surprise. The conventional ViT-vs-CNN story is "ViT needs lots of data"  but our results show the opposite trend on EuroSAT. One of the likely reasons are we're not training ViT from scratch, we're fine-tuning an ImageNet-pretrained ViT. The pretraining absorbs the data-hungriness, and what's left is a strong representation that transfers efficiently to satellite imagery. Meanwhile, at full data, ResNet18's convolutional inductive bias (locality, translation equivariance) starts to pay off.

## Sample Efficiency of the Dual Branch Models

To evaluate sampling efficiency, we evaluated each of the models on the 10% of data that the 90/10 train/split models did not see during their training.

The sample efficiency of the ResNet model is close to what is expected, and similar to the more traditional approaches we tried (see the Sampling Efficiency section below for more detail). However, the ConvNeXt had impressive performance across the board, but especially at very low training sizes. Even training it on only 270 labeled images from EuroSAT, it was able to achieve an impressive 92.70% test accuracy.

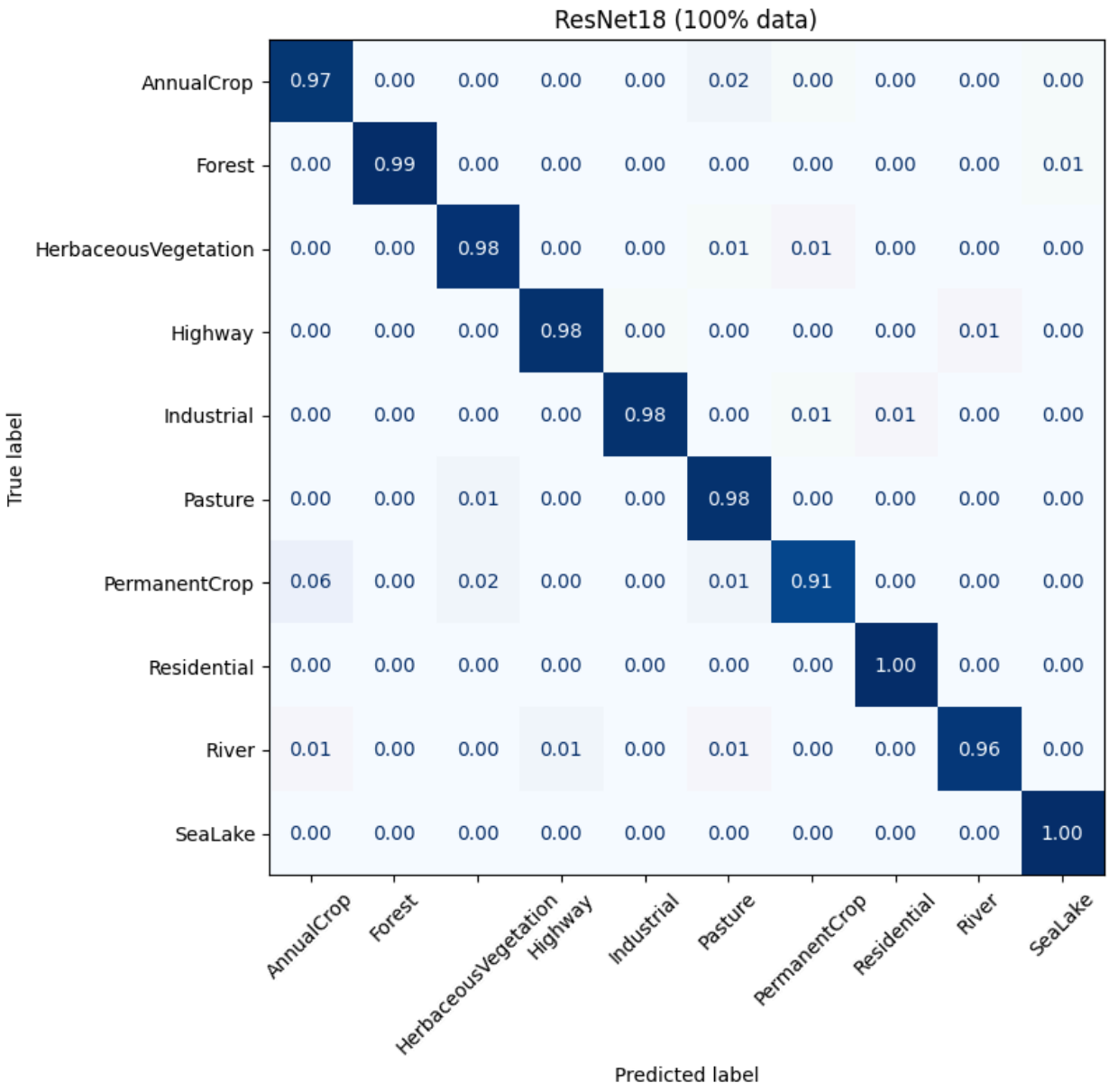
| Fraction | Dual-Branch ResNet18 | Dual-Branch ConvNeXt |
|---|---|---|
| 1% | 85.78% | 92.70% |
| 5% | 95.41% | 96.22% |
| 10% | 97.56% | 98.26% |
| 25% | 98.44% | 98.63% |
| 50% | 98.59% | 99.19% |
| 80% | 98.93% | 99.33% |
| 90% | 99.15% | 99.33% |




# Where Do They Disagree?

## Confusion Matrices

Both models hit ~98% overall, but where exactly do they make their mistakes?


ResNet18 (100% data)
True label
Predicted label
AnnualCrop
Forest
HerbaceousVegetation
Highway
Industrial
Pasture
PermanentCrop
Residential
River
SeaLake
0.97 0.00 0.00 0.00 0.00 0.02 0.00 0.00 0.00 0.00
0.00 0.99 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.01
0.00 0.00 0.98 0.00 0.00 0.01 0.01 0.00 0.00 0.00
0.00 0.00 0.00 0.98 0.00 0.00 0.00 0.00 0.01 0.00
0.00 0.00 0.00 0.00 0.98 0.00 0.01 0.01 0.00 0.00
0.00 0.00 0.01 0.00 0.00 0.98 0.00 0.00 0.00 0.00
0.06 0.00 0.02 0.00 0.00 0.01 0.91 0.00 0.00 0.00
0.00 0.00 0.00 0.00 0.00 0.00 0.00 1.00 0.00 0.00
0.01 0.00 0.00 0.01 0.00 0.01 0.00 0.00 0.96 0.00
0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 1.00


The most interesting per-class gap is **Pasture.**

ResNet18 nails it at 98%, but ViT-Small drops to 90%, confusing pasture with forest and herbaceous vegetation. The likely culprit is ViT's patch tokenization. Chopping the image into 16×16 blocks loses fine-grained texture information, and grass-vs-canopy-vs-shrubs is exactly the kind of distinction that lives in micro-texture. CNNs catch this naturally through local convolutions.

Both models share one weakness distinguishing **PermanentCrop from AnnualCrop and HerbaceousVegetation** (~91-92% accuracy). However, this is

fundamentally a data problem. Single-snapshot RGB tiles just don't carry enough information to tell crop types apart without temporal or multispectral data.

So ResNet and ViT have complementary strengths. ViT wins on sample efficiency and most classes; ResNet wins on fine-texture vegetation. An ensemble would probably outperform either alone — something we'd love to try with more time.

# References


[1] Eurosat: A novel dataset and deep learning benchmark for land use and land cover classification. Patrick Helber, Benjamin Bischke, Andreas Dengel, Damian Borth. IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing, 2019.

[2] Introducing EuroSAT: A Novel Dataset and Deep Learning Benchmark for Land Use and Land Cover Classification. Patrick Helber, Benjamin Bischke, Andreas Dengel. 2018 IEEE International Geoscience and Remote Sensing Symposium, 2018.

[3] Yi Yang and Shawn Newsam, "Bag-Of-Visual-Words and Spatial Extensions for Land-Use Classification," ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems (ACM GIS), 2010.

[4] Sentinel-2 Level-2A. European Space Agency. Microsoft Planetary Computer, 2026. Available: https://planetarycomputer.microsoft.com/dataset/sentinel-2-l2a

[5] Report on the Environment: Land Use. U.S. Environmental Protection Agency. U.S. Environmental Protection Agency, 2026. Available: https://www.epa.gov/report-environment/land-use

[6] Deep Residual Learning for Image Recognition. Kaiming He, Xiangyu Zhang, Shaoqing Ren, Jian Sun. IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016.

[7] ImageNet: A Large-Scale Hierarchical Image Database. Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, Li Fei-Fei. IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2009.

[8] A ConvNet for the 2020s. Zhuang Liu, Hanzi Mao, Chao-Yuan Wu, Christoph Feichtenhofer, Trevor Darrell, Saining Xie. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2022.

[9] An Image is Worth 16×16 Words: Transformers for Image Recognition at Scale.
Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, et al.
International Conference on Learning Representations (ICLR), 2021.

[10] Ross Wightman. PyTorch Image Models (timm). GitHub repository, 2019.
Available: https://github.com/huggingface/pytorch-image-models